



Final SAST and DAST Security Testing Report

Proprietary Statement

(This document contains information from TestingXperts that is highly confidential and privileged. The information is intended for the personal use of TestingXperts)



Document Details:

Company	TestingXperts
Document Title	Tx-Catalyst Final SAST and DAST Retest Rport
Submission Date	30-April-2026
Classification	Highly Confidential
Document Type	Report

Document History:

Version	Date	Author(s)	Comments
1.0	28 th Apr'26	Security Team	Final SAST and DAST Report
2.0	30 th Apr'26	Prasanth, Shrey	Final SAST and DAST Retest Report

Contents

1. Executive Summary	4
1.1 Test Timeline	4
1.2 Approach	5
1.3 Activities	5
1.4 Summary of Findings	5
2. Vulnerabilities Description	9
2.1 Weak Password Requirements	9
2.2 Encryption Not Enforced	11
2.3 No Account Lockout	12
2.4 Account Enumeration	14
2.5 Use of Hard-coded Credentials	16
2.6 Missing Cookie 'Flags'	17
2.7 Lack of Post-Quantum Cryptography (PQC) Support	19
2.8 Insecure Content Security Policy (CSP)	20
2.9 Unrestricted Cross-Origin Resource Sharing (CORS)	22
3. TestingXperts Methodology	24
4. Conclusion	27

1. Executive Summary

“TestingXperts” conducted a Static and Dynamic Application Security Testing assessment against their application “Tx_Catalyst”. The objective of the test was to meet information security standards like OWASP and NIST and to simulate real-world attack scenarios and aims to exploit vulnerabilities, identify design flaws, leverage combination of lower impact flaws into higher impact vulnerabilities, and determine if identified flaws affect the confidentiality, integrity, or availability of the application.

Following target URL were in scope: -

<http://192.168.3.90:3002>

https://irsyepgmxsjwgpnzowvn.supabase.co/auth/v1/token?grant_type=password

As part of this Testing, we tried to capture the business logic flow of the application, list down the potential threats to the application, and prepare this Security test report.

Our Approach was to meet Information Security Industry standards like OWASP Top 10, and NIST.

80% – 85% attacks in the real world are based on these techniques. Additionally, we make an effort to perform attacks relevant to business logic.

During this engagement, we primarily focused on **Authentication and Authorization mechanism, Session Management, Input Validation, Security Misconfiguration, Cryptography, Business Logic bypasses, Server and Client-side attacks, etc.**

1.1 Test Timeline

Security testing on web application was performed during VAPT engagement on 28th Apr 2026.

Retesting of the web application was performed on 30th Apr 2026.

1.2 Approach

Activities
Application Understanding, Information Gathering and Identifying Application Entry Points
Understanding of the Source Code and Dependencies
Vulnerability Scanning and Manual Verification
Manual Penetration Testing as per OWASP and NIST Methodologies
Vulnerability Analysis
Removal of False Positive
Report Preparation

1.3 Activities

The following set of security testing activities are performed by the team:

- Understanding of the Source Code and Dependencies used for the platform.
- Understanding of the applications, security requirements, security assets.
- Understanding of user roles, access levels, authentications and Business Logic.
- Vulnerability Scanning and Penetration Testing as per OWASP 10 and OSSTMM standards using TX's customized security testing framework (based on open-source tools and Burp Suite Pro).
- Determine possible extent access or impact to the system by attempting to exploit vulnerabilities.
- Vulnerability Analysis to remove 'false positives' and identify potential remedies.
- Provide tactical recommendations to address security issues of immediate consequence.
- Provide strategic recommendations to enhance security by leveraging industry best practice

1.4 Summary of Findings

TestingXperts Security Team discovered **9** vulnerabilities during the course of SAST and DAST analysis. The report classifies each vulnerability in appropriate categories, and we have suggested mitigation strategy for them.

Going ahead, TestingXperts Security Team recommends:

1. Address and remediate the vulnerabilities identified through SAST in accordance with secure coding best practices across the development lifecycle.
2. Fixing of reported vulnerabilities.
3. Follow the Iterative Secure SDLC process, this includes complete security regression test with any new component that is added.

Vulnerabilities By Severity

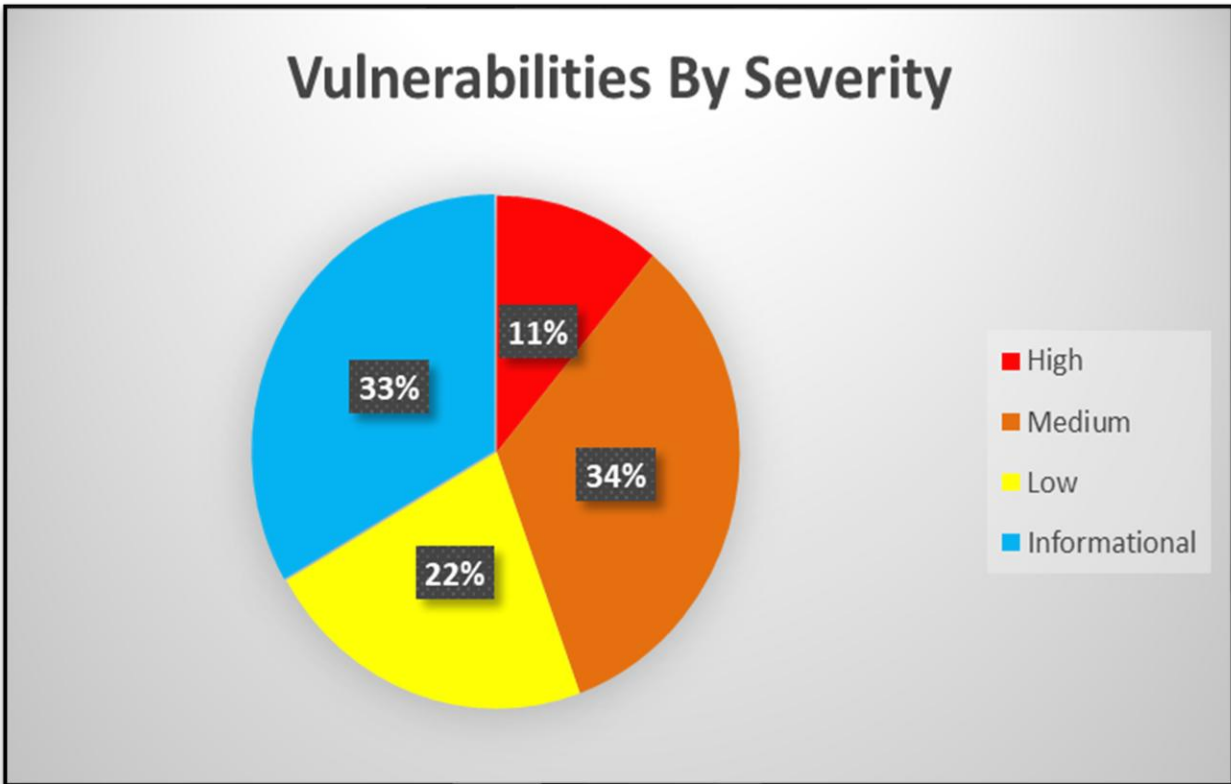
The following vulnerabilities were identified during the engagement: -

Sr. No.	Vulnerability Name	Severity	CVSS Score
1.	Weak Password Requirements	High	8.2
2.	Encryption Not Enforced	Medium	6.5
3.	No Account Lockout	Medium	5.3
4.	Account Enumeration	Medium	5.3
5.	Use of Hard-coded Credentials	Low	3.7
6.	Missing Cookie 'Flags'	Low	3.1
7.	Lack of Post-Quantum Cryptography (PQC) Support	Informational	0.0
8.	Insecure Content Security Policy (CSP)	Informational	0.0
9.	Unrestricted Cross-Origin Resource Sharing (CORS)	Informational	0.0

Tabular Summary

Total Number of Vulnerabilities	9
Critical Severity Vulnerabilities	0
High Severity Vulnerabilities	1
Medium Severity Vulnerabilities	3
Low Vulnerabilities	2
Informational Vulnerabilities	3

Classification of Vulnerabilities



"As pie chart indicates that 11% vulnerabilities are identified as High, 34% vulnerabilities are identified as Medium, 22% as Low and 33% vulnerabilities are acknowledged as Informational."

2. Vulnerabilities Description

High Severity Vulnerabilities

2.1 Weak Password Requirements

Vulnerability Information:

The application allows passwords which are considered to be weak by NIST guidelines.

Exploit Description:

Attackers who have a list of valid usernames will often try to use dictionary-style attacks to force entry into the system. The weaker the password, the higher the probability of a successful attack.

Severity Description:

Access to the system with the permissions/access privileges of the compromised account.

Vulnerability Severity:

- Risk - High
- Impact - High
- Exploitability - High
- CVSS Score: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N = 8.2 (High)

Category: Authentication

Instance:

https://irsyepgmxsjwgnzowvn.supabase.co/auth/v1/token?grant_type=password

Proof-Of-Concept:

Provided is the request/response from the Burp Suite tool, which shows that the application accepts weak passwords.

Vulnerable Request/Response:

Request:

```
POST /auth/v1/signup HTTP/2
Host: irsyepgmxsjwgnzowvn.supabase.co
Content-Length: 266
Sec-Ch-Ua-Platform: "Windows"
Authorization: Bearer eyJhbG[Redacted for Brevity]
X-Supabase-API-Version: 2024-01-01
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
```

```
Sec-Ch-Ua-Mobile: ?0
X-Client-Info: supabase-ssr/0.9.0 createBrowserClient
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Content-Type: application/json;charset=UTF-8
Apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Imlyc
3llcGdteHNqd2dwbnpvd3ZuIiwicm9sZSI6ImFub24iLCJpYXQiOiJlZ3NzczOTIyMzYsImV4cCI
6MjA5Mjk2ODIzNn0.2oxaTGacG1bsh8YjVZnVKoP-EfpGojEQltUXsFluRdc
Accept: */*
Origin: http://192.168.3.90:3002
[Redacted for Brevity]
```

```
{"email":"Demo@yopmail.com","password":"123456","data":{"full_name":"Demo"
},"gotrue_meta_security":{},"code_challenge":"ed600f97de279d21660bc2b41363
72bfae4228a59df0be2c75322ad70dbd8440eb8ca8d9cd04f0bb64ca48fb1d55911495e3bf
e76e692164","code_challenge_method":"plain"}
```

Response:

HTTP/2 200 OK

```
Date: Thu, 30 Apr 2026 09:01:26 GMT
Content-Type: application/json
Cf-Ray: 9f455c9e59a3a6f5-DEL
Cf-Cache-Status: DYNAMIC
Access-Control-Allow-Origin: http://192.168.3.90:3002
Server: cloudflare
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Vary: Origin, Accept-Encoding
[Redacted for Brevity]
```

Remediation:

Authentication mechanisms should always require strong passwords. NIST guidelines recommend against using complexity rules, and instead rely on password length and checks against common password lists to ensure that users choose strong passwords.

Consider the following when creating a password policy: -

- Minimum of 8 characters.
- Disallow passwords that match commonly used / leaked passwords, and words relating to the application or company.
- Allow passwords up to at least 64 characters, so that users can create secure passphrases.
- Allow all printable ASCII characters, including spaces, and unicode characters in passwords.

See also: <https://pages.nist.gov/800-63-3/sp800-63b.html>

Medium Severity Vulnerabilities

2.2 Encryption Not Enforced

Vulnerability Information:

During the application test, it was detected that the site uses an encrypted connection to protect sensitive information. However, it was possible to receive these resources using HTTP, which means that sensitive information may be sent unencrypted to the server and/or back to the user.

Exploit Description:

An attacker attempting to monitor or manipulate transmitted data would seek to actively reroute data traffic or passively observe clear-text communications. A number of techniques could be employed to accomplish this including ARP spoofing, DNS spoofing, route manipulation, and switch monitoring. The ease of exploit is generally related to the proximity of the attacker to the victim.

Severity Description:

All application data including user credentials could be manipulated and observed by an attacker. User credentials are often reused across disparate systems which extends the reach of the compromise. The confidentiality and integrity of application data could be undermined.

Vulnerability Severity:

- Risk - Medium
- Impact - Medium
- Exploitability - Low
- CVSS Score: - CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:N = 6.5 (Medium)

Category: Configuration

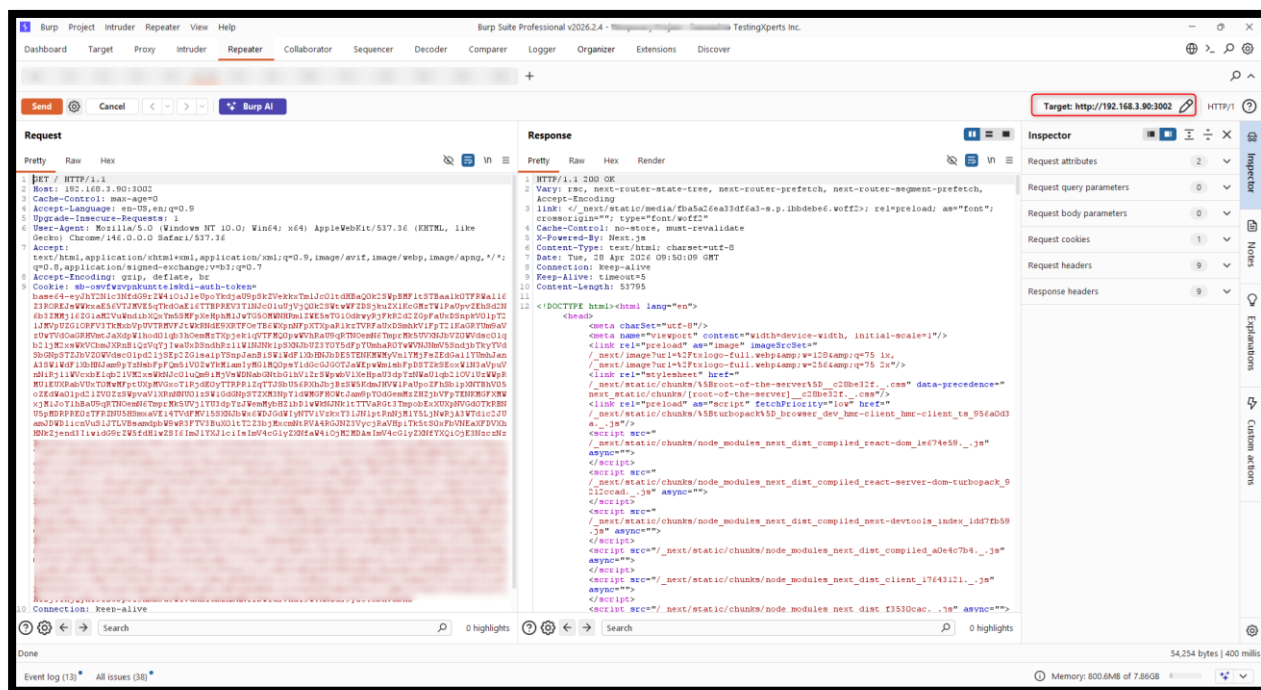
Instance:

<http://192.168.3.90:3002/>

Proof-Of-Concept:

Provided is the screenshot which demonstrate that the application does not enforce encryption.

Screenshot:



Remediation:

Applications should use transport-level encryption (SSL/TLS) to protect all communications passing between the client and the server. The Strict-Transport-Security HTTP header should be used to ensure that clients refuse to access the server over an insecure connection.

2.3 No Account Lockout

Vulnerability Information:

The application does not keep count of the number of failed logins attempts for an account therefore allowing for the possibility that an attacker can repeatedly guess a user's password in an attempt to brute-force access to an individual user's account. This attack can be easily automated by a malicious user or malware.

Exploit Description:

Without an account lockout, an attacker can continue to guess a user's password until it is found, and the account is compromised.

Severity Description:

A scripted attack could result in discovery of the authenticated users password in addition to changing his/her password and effectively locking them out of the application while granting the attacker access to the application.

Vulnerability Severity:

- Risk - Medium
- Impact - Medium
- Exploitability - High
- CVSS Score: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N= 5.3 (Medium)

Category: Authentication

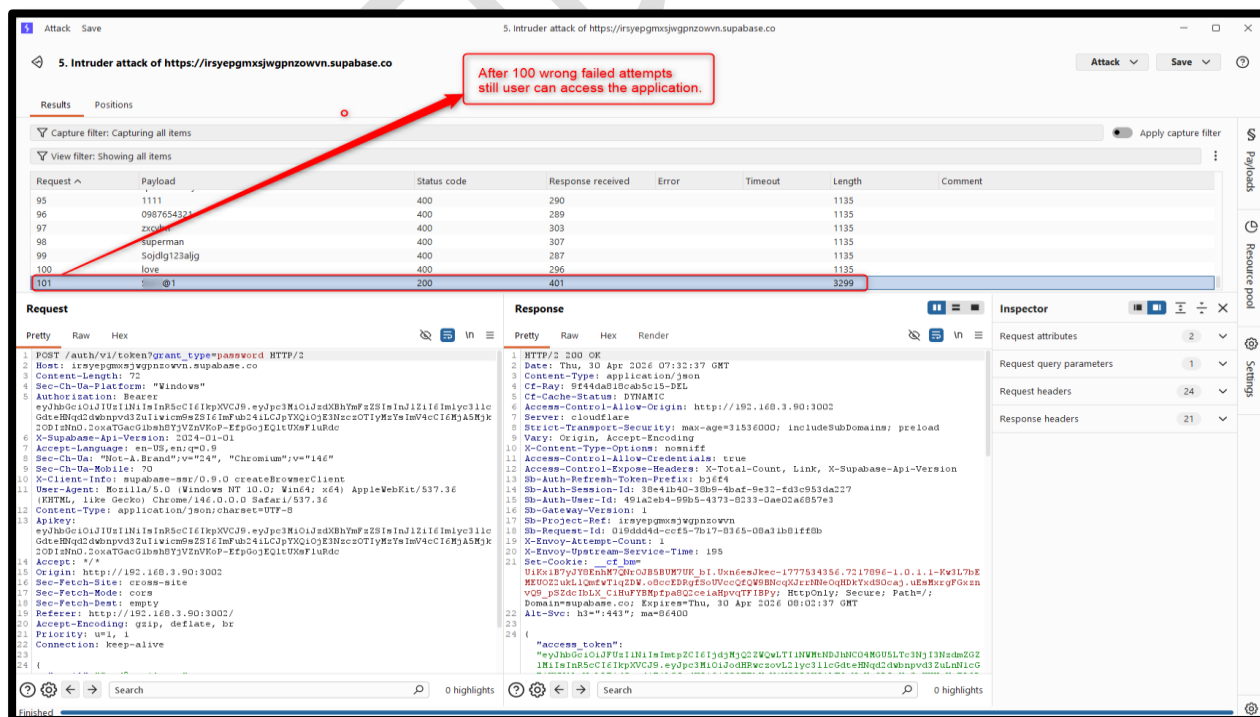
Instance:

<https://irsyepgmxsjwgpznzown.supabase.co/auth/v1/signup>

Proof-Of-Concept:

Provided is the screenshot from Burp Suite Intruder, which demonstrates that even after 100 incorrect attempts, the application continues to accept password attempts on the 101st attempt, indicating the absence of an effective account lockout mechanism.

Screenshot:



The screenshot displays the Burp Suite Intruder interface for an attack on the endpoint `https://irsyepgmxsjwgpznzown.supabase.co`. The 'Results' tab shows a table of 101 requests. The first 100 requests (indices 95-100) all resulted in a 400 status code, indicating failed login attempts. The 101st request (index 101) resulted in a 200 status code, indicating a successful login. A red arrow points to the 101st request, and a text box states: "After 100 wrong failed attempts still user can access the application."

The 'Request' tab shows the details of the 101st request, which is a POST to `/auth/v1/token?grant_type=password`. The 'Response' tab shows the response, which is a 200 OK status with a JSON body containing an `access_token`.

Remediation:

Track the number of failed log in attempts for a user and mark the account as locked when the number of consecutive unsuccessful login attempts reaches a reasonable threshold (typically 3 to 5 failed attempts in a row). Require a user to go through the forgotten password process to remove the lock on their account.

2.4 Account Enumeration

Vulnerability Information:

The application returns different responses to the user depending on whether the username or email address provided in the request is valid or invalid.

Exploit Description:

When submitting a valid username & invalid password combination 'X' amount of times, the server responds back with a message indicating the account has been locked out: (See exhibits) When entering and submitting an invalid username & password combination the same number of attempts no account lockout message is returned. The difference in responses allows an attacker to determine legitimate accounts on the application.

Severity Description:

Using automated attacks, exploitation may result in the unintentional yielding of valid account names or login ID's that exist within the application. These ID's may then be used in additional attacks such as spamming, phishing, DoS, Bruteforce Logins etc. The effort is moderate because of the time required to brute for the desired information or even to use large black market lists against the system to harvest valid account logins. The attack can be detected due to the repeated nature of the attack.

Vulnerability Severity:

- **Risk** - Medium
- **Impact** - Medium
- **Exploitability** - High
- **CVSS Score:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N= **5.3 (Medium)**

Category: Data Exposure

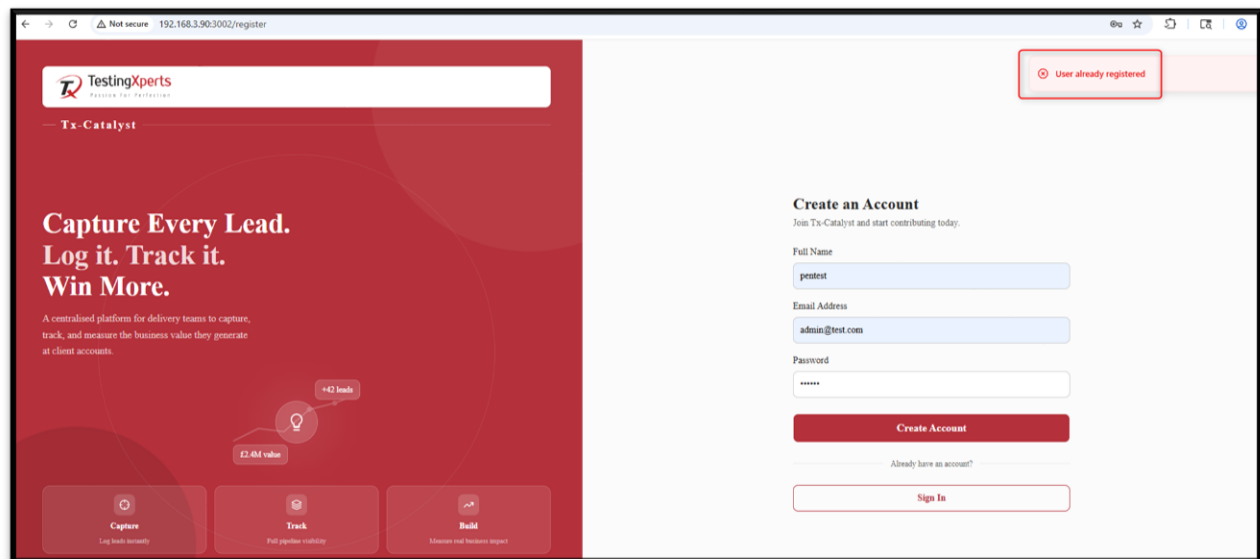
Instance:

<http://192.168.3.90:3002/login>

Proof-Of-Concept:

Provided is the screenshot that demonstrates that username or account enumeration is possible on the login page.

Screenshot:



Remediation:

The application should avoid disclosing whether a user account already exists during the registration process. Instead of returning specific error messages such as “User already registered”, the system should respond with a generic and consistent message for all registration attempts, regardless of whether the email is already in use.

Additionally, the application should implement rate limiting and CAPTCHA controls to prevent automated enumeration attacks. Proper validation and secure handling of user input must be maintained to ensure no sensitive information is exposed through responses.

Optionally, after successful registration, the application may securely authenticate and redirect the user to the dashboard, ensuring session management is handled in a secure manner.

Low Severity Vulnerabilities

2.5 Use of Hard-coded Credentials

Vulnerability Information:

Identification and Authentication Failures often occur when sensitive information is embedded directly into the application's source code. During the security assessment, it was observed that credentials (such as passwords or authentication keys) are hard-coded in plain text within the codebase rather than being managed through secure, externalized configurations.

Exploit Description:

Since strings can be easily extracted from application source code or binaries, an attacker with access to the codebase can identify and retrieve these credentials. Hard-coding sensitive information is particularly risky for applications that are widely distributed or open-source. Once these credentials are recovered, they can be used to gain unauthorized access to sensitive components, including databases, file storage, APIs, or other internal services.

Severity Description:

The presence of hard-coded credentials poses a significant risk as it simplifies the process for an intruder to compromise the system. Unlike dynamic credentials, these cannot be easily rotated without a code change and deployment. If these credentials provide access to production databases or employee lists, the impact on data confidentiality is substantial.

Vulnerability Severity:

- **Risk** - Low
- **Impact** - High
- **Exploitability** - High
- **CVSS Score**: - CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N = **3.7 (Low)**

Category: Cryptography

Instance:

[backend/scripts/import_employees.py](#)

Proof-Of-Concept:

Provided is the screenshot of the SAST analysis tool which demonstrates that credentials are hardcoded in the application.

Screenshot:

```
backend/scripts/import_employees.py [icon] Open in IDE

32 sys.path.insert(0, str(BACKEND_DIR))
33
34 from app.database.supabase import get_supabase_admin # noqa: E402
35
36 logging.basicConfig(level=logging.INFO, format="% (asctime)s %(levelname)s %(message)s")
37 logger = logging.getLogger("import_employees")
38
39 CSV_PATH = (
40     BACKEND_DIR.parent / "Delivery Project-Emp List(Delivery Emp List).csv"
41 )
42 SHARED_PASSWORD = "Txcatalyst@123"

"password" detected here, review this potentially hard-coded credential.
```

Remediation:

- Sensitive credentials should not be hardcoded in the source code. Instead, they must be securely stored outside the codebase, such as in configuration files that are excluded from version control.
- Credentials should be managed using secure storage mechanisms such as encrypted databases or dedicated secret management solutions.
- It is recommended to leverage cloud-based secret management services (e.g., AWS Secrets Manager, Azure Key Vault, Google Secret Manager) to ensure secure storage, access control, and rotation of secrets.
- In case credentials have been exposed within the source code or repository, they must be immediately revoked and rotated, and any affected systems should be reviewed for potential misuse.

2.6 Missing Cookie 'Flags'

Vulnerability Information:

The application sets cookies without enforcing appropriate security attributes such as 'Secure', 'HttpOnly', and 'SameSite'. The absence of these attributes increases the risk of cookies being accessed via client-side scripts, transmitted over unencrypted channels, or included in cross-site requests.

Exploit Description:

An attacker may exploit missing cookie attributes by intercepting or manipulating cookies during transmission or execution within the browser. Without the 'Secure' flag, cookies can be transmitted over unencrypted HTTP connections, making them susceptible to interception via man-in-the-middle attacks. In the absence of the 'HttpOnly' attribute, cookies can be accessed

through client-side scripts, enabling attackers to steal session identifiers through cross-site scripting attacks. Additionally, without the 'SameSite' attribute, cookies may be included in cross-origin requests, increasing the risk of cross-site request forgery attacks.

Severity Description:

Exploitation of missing cookie security attributes may result in unauthorized access to user sessions, leakage of sensitive information, and potential account compromise. Attackers may leverage these weaknesses to hijack authenticated sessions or perform actions on behalf of legitimate users.

Vulnerability Severity:

- Risk - Low
- Impact - Low
- Exploitability - Low
- CVSS Score: - CVSS v3.1 Vector: AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N = **3.1 (Low)**

Category: Session Management

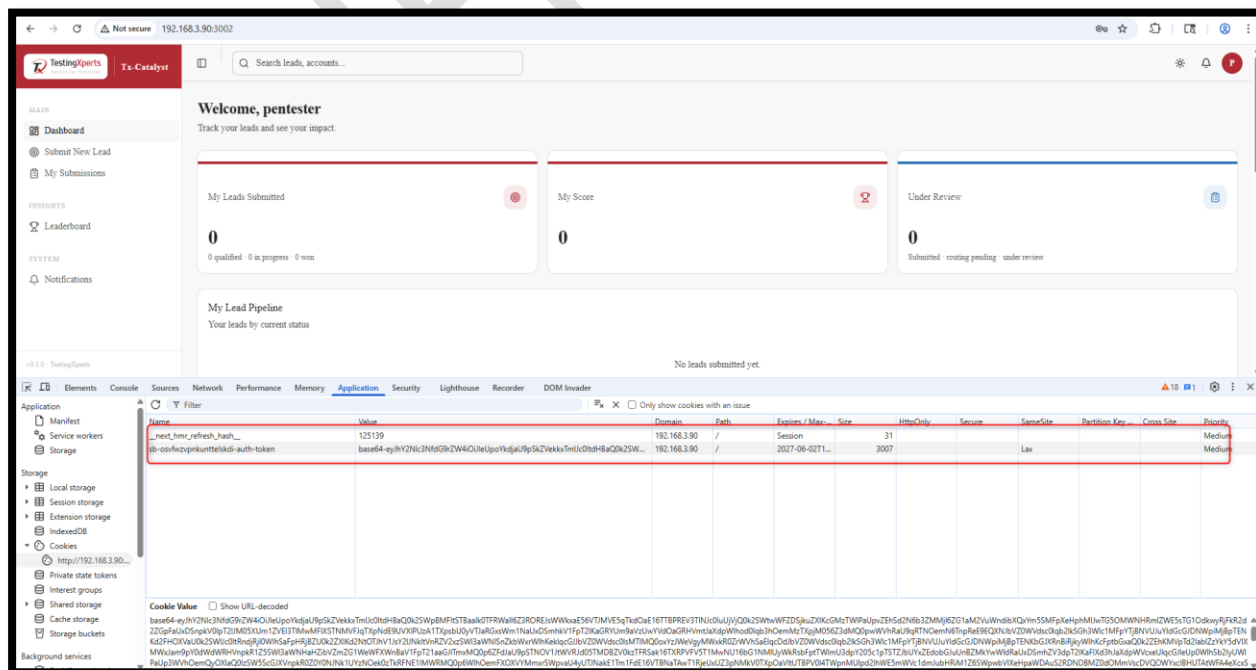
Instance:

<http://192.168.3.90:3002/>

Proof-Of-Concept:

Provided is the screenshot which demonstrate that the application is missing cookies attributes.

Screenshot:



Remediation:

Ensure that all sensitive cookies are configured with the appropriate security attributes as outlined in industry best practices. This includes:

- Setting the 'Secure' flag to ensure cookies are only transmitted over HTTPS connections.
- Setting the 'HttpOnly' flag to prevent client-side scripts from accessing cookie values.
- Implementing the 'SameSite' attribute (e.g., Strict or Lax) to mitigate cross-site request forgery risks.

Informational Vulnerabilities

2.7 Lack of Post-Quantum Cryptography (PQC) Support

Vulnerability Information:

The application does not fully support modern Post-Quantum Cryptography (PQC) algorithms as recommended by NIST, which may pose a risk against future quantum computing attacks. Current implementation primarily relies on classical cryptographic algorithms (e.g., RSA/ECDSA), which are considered vulnerable to "Harvest Now, Decrypt Later" attacks in the long term.

Exploit Description:

An attacker with the capability to capture encrypted traffic today may store it and decrypt it in the future once quantum computing becomes viable, due to the absence of strong PQC mechanisms.

Severity Description:

This could result in long-term exposure of sensitive information such as authentication tokens, session data, and confidential communications.

Vulnerability Severity: Informational

Category: Cryptography

Instance:

<http://192.168.3.90:3002/>

Proof-Of-Concept:

Provided are the outputs from the Post-Quantum Cryptography detection tool, which demonstrate that the application is not fully quantum-ready and lacks support for multiple NIST-recommended PQC algorithms.

Screenshot:

TARGET 192.168.3.90	PORTS SCANNED 443	TLS VERSION —	CIPHER SUITE —	ELAPSED 6.23s	TIMESTAMP 2026-04-28 11:34:11 UTC
------------------------	----------------------	------------------	-------------------	------------------	--------------------------------------

NIST PQC GROUP ANALYSIS					
ML-KEM X NOT FOUND FIPS 203 - Key Encapsulation Mechanism <i>née née CRYSTALS-Kyber</i> No CRYSTALS-Kyber / ML-KEM implementations detected on this target. ML-DSA X NOT FOUND FIPS 204 - Lattice-Based Digital Signature <i>née née CRYSTALS-Dilithium</i> No CRYSTALS-Dilithium / ML-DSA implementations detected on this target. SLH-DSA X NOT FOUND FIPS 205 - Hash-Based Digital Signature <i>née née SPHINCS+</i> No SPHINCS+ / SLH-DSA implementations detected on this target. FN-DSA X NOT FOUND FIPS 206 - Compact Lattice Signature <i>née née FALCON</i> No FALCON / FN-DSA implementations detected on this target.					
FULL DETECTION SUMMARY					
GROUP	STANDARD	PURPOSE	FORMERLY KNOWN AS	STATUS	VARIANTS DETECTED
ML-KEM	FIPS 203	Key Encapsulation Mechanism	CRYSTALS-Kyber	X NOT FOUND	—
ML-DSA	FIPS 204	Lattice-Based Digital Signature	CRYSTALS-Dilithium	X NOT FOUND	—
SLH-DSA	FIPS 205	Hash-Based Digital Signature	SPHINCS+	X NOT FOUND	—
FN-DSA	FIPS 206	Compact Lattice Signature	FALCON	X NOT FOUND	—

Remediation:

- Implement NIST-approved Post-Quantum Cryptographic algorithms such as:
 - ML-KEM (Key Exchange)
 - ML-DSA / SLH-DSA / FN-DSA (Digital Signatures)
- Adopt hybrid cryptographic models combining classical + PQC algorithms.
- Ensure TLS configurations support PQC-enabled cipher suites.
- Continuously monitor NIST PQC standardization updates and upgrade cryptographic implementations accordingly.

2.8 Insecure Content Security Policy (CSP)

Vulnerability Information:

The Content Security Policy (CSP) set by the application contained policy configurations which potentially open the application to a number of attacks.

Exploit Description:

The exploitability of this flaw depends on whether an attacker can inject JavaScript into the application, or whether any allowed script sources can be abused to host potentially malicious files.

Severity Description:

If an attacker is able to take advantage of the CSP misconfigurations, they may be able to bypass it and run malicious JavaScript, which could lead to partial compromise of confidentiality and integrity.

Vulnerability Severity: Informational

Category: Configuration

Instance:

<http://192.168.3.90:3002/notifications>

Proof-Of-Concept:

Provided are the request/response which demonstrate the use of insecure Content Security Policy (CSP) directives, including 'unsafe-eval' within script-src and 'unsafe-inline' within style-src, indicating a weak CSP configuration.

Vulnerable Request/Response:

Request:

```
GET /api/notifications/count HTTP/1.1
Host: 192.168.3.90:8005
Authorization: Bearer eyJhbGciOiJ[Redacted for Brevity]
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://192.168.3.90:3002
Referer: http://192.168.3.90:3002/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

Response:

```
HTTP/1.1 200 OK
date: Thu, 30 Apr 2026 10:42:25 GMT
server: uvicorn
content-length: 12
content-type: application/json
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera=(), microphone=(), geolocation=()
strict-transport-security: max-age=31536000; includeSubDomains
content-security-policy: default-src 'self'; script-src 'self'
'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline'; img-
```

```
src 'self' data: blob;; font-src 'self' data;; connect-src 'self'  
https://*.supabase.co wss://*.supabase.co; frame-ancestors 'none';  
access-control-allow-credentials: true  
access-control-allow-origin: http://192.168.3.90:3002  
vary: Origin  
  
{"unread":0}
```

Remediation:

Ensure that the Content Security Policy does not contain weak policy configurations, or sources known to host files which can be used to bypass the CSP. Consider the risk associated with allowing "unsafe-inline" and "unsafe-eval" policies

2.9 Unrestricted Cross-Origin Resource Sharing (CORS)

Vulnerability Information:

The application server does not restrict the acceptable domain(s) allowed to make cross-origin requests.

Exploit Description:

An attack, with MiTM (Man-in-the-Middle) access, could inject malicious requests that would then be served to the client.

Severity Description:

Any malicious payload injected via this attack vector would appear to be delivered directly from the trusted site.

Vulnerability Severity: Informational

Category: Configuration

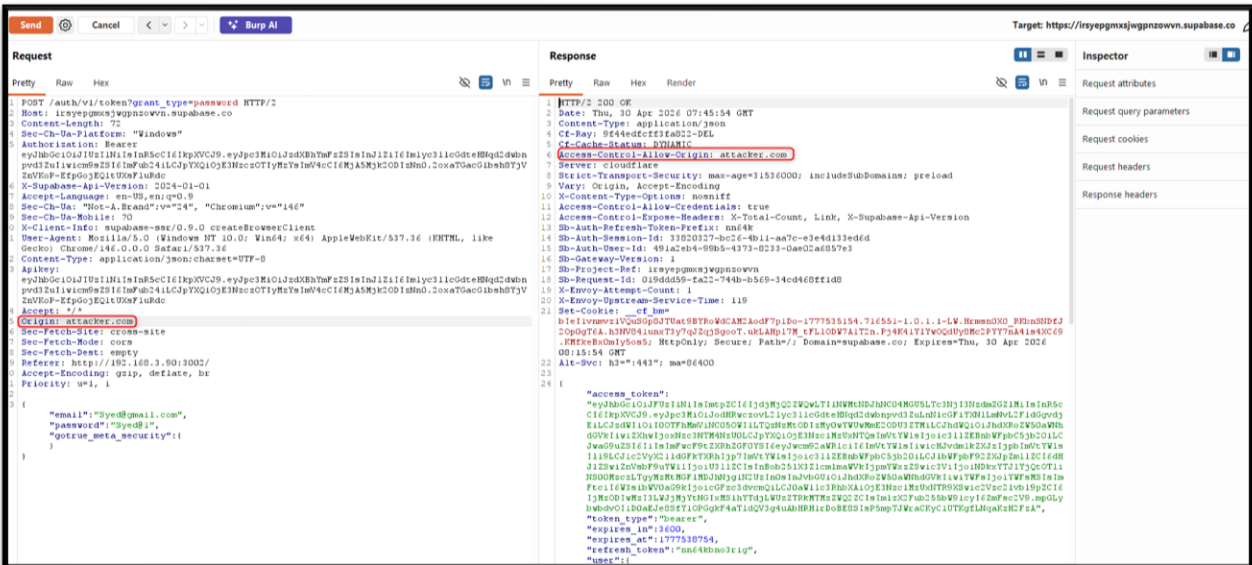
Instance1:

https://irsyepgmxsjwgpznzowvn.supabase.co/auth/v1/token?grant_type=password

Proof-Of-Concept:

Provided is the screenshot which demonstrates an insecure Cross-Origin Resource Sharing (CORS) configuration, where overly permissive CORS headers allow unauthorized cross-origin access to application resources

Screenshot:



Remediation:

The Access-Control-Allow-Origin header should be restricted to allow only acceptable/trusted domain(s).

3. TestingXperts Methodology

Software Application Security Testing (SAST) and Software Composition Analysis (SCA) is conducted to observe the application code in a run-time environment and to simulate real-world attack scenarios. Automated testing can identify design flaws, evaluate environmental conditions, compound multiple lower risk flaws into higher risk vulnerabilities, and determine if identified flaws affect the confidentiality, integrity, or availability of the application.

Objectives:-

The stated objectives of a SAST and SCA are:

- Perform testing, using proprietary and/or public tools, to determine whether it is possible for an attacker to:
 - Circumvent authentication and authorization mechanisms
 - Escalate application user privileges
 - Hijack accounts belonging to other users
 - Violate access controls placed by the site administrator
 - Alter data or data presentation
 - Corrupt application and data integrity, functionality and performance
 - Circumvent application business logic
 - Circumvent application session management
 - Break or analyse use of cryptography within user accessible components
 - Determine possible extent access or impact to the system by attempting to exploit vulnerabilities
- Score vulnerabilities using the Common Vulnerability Scoring System (CVSS v3.1)
- Provide tactical recommendations to address security issues of immediate consequence
- Provide strategic recommendations to enhance security by leveraging industry best practices Attack vectors

In order to achieve the stated objectives, the following tests are performed as part of the automated and manual vulnerability assessment and penetration testing, when applicable to the platforms and technologies in use:

- Cross Site Scripting (XSS)
- SQL Injection
- Command Injection
- Cross Site Request Forgery (CSRF)
- Authentication/Authorization Bypass
- Session Management testing, e.g. token analysis, session expiration, and logout effectiveness
- Account Management testing, e.g. password strength, password reset, account lockout, etc.

- Directory Traversal
- Response Splitting
- Stack/Heap Overflows
- Format String Attacks
- Cookie Analysis
- Server Side Includes Injection
- Remote File Inclusion
- LDAP Injection
- XPATH Injection
- Internationalization attacks
- Denial of Service testing at the application layer only
- AJAX Endpoint Analysis
- Web Services Endpoint Analysis
- HTTP Method Analysis
- SSL Certificate and Cipher Strength Analysis
- Forced Browsing

Risk Rating Methodology

TestingXperts uses a CVSS V 3.1 calculator to determine the severity of the identified vulnerabilities. We take into account the attack complexity, attack vector, level of privileges, user interaction and the impact on Confidentiality, Integrity and Availability of the application to determine the severity of the vulnerability.

The CVSS assessment measures three areas of concern:

- Base Metrics for qualities intrinsic to a vulnerability
- Temporal Metrics for characteristics that evolve over the lifetime of vulnerability
- Environmental Metrics for vulnerabilities that depend on an implementation or environment

Along with the CVSS score we provide Risk Rating for the vulnerabilities as well, the Risk Rating is TestingXperts recommended prioritization for addressing findings.

Based on the severity of the vulnerability, they are assigned below ratings:

CVSS Severity Rating	CVSS Score
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Informational	0.0

Overall Risk

Overall risk reflects TestingXperts estimation of the risk that a finding poses to the target system or systems have. It takes into account the impact of the finding, the difficulty of exploitation, and any other relevant factors.

Critical -Implies an immediate, easily accessible threat of total compromise.

High- Implies an immediate threat of system compromise, or an easily accessible threat of large-scale breach.

Medium- A difficult to exploit threat of large-scale breach, or easy compromise of a small portion of the application.

Low- Implies a relatively minor threat to the application.

Informational- No immediate threat to the application. May provide suggestions for application improvement, functional issues with the application, or conditions that could later lead to an exploitable finding.

Impact

Impact reflects the effects that successful exploitation upon the target system or systems. It takes into account potential losses of confidentiality, integrity and availability, as well as potential reputational losses.

High - Attackers can read or modify all data in a system, execute arbitrary code on the system, or escalate their privileges to superuser level.

Medium - Attackers can read or modify some unauthorized data on a system, deny access to that system, or gain significant internal technical information.

Low - Attackers can gain small amounts of unauthorized information or slightly degrade system performance. May have a negative public perception of security.

Exploitability

Exploitability reflects the ease with which attackers may exploit a finding. It takes into account the level of access required, availability of exploitation information, requirements relating to social engineering, race conditions, brute forcing, etc, and other impediments to exploitation.

High - Attackers can unilaterally exploit the finding without special permissions or significant roadblocks.

Medium- Attackers would need to leverage a third party, gain non-public information, exploit a race condition, already have privileged access, or otherwise overcome moderate hurdles in order to exploit the finding.

Low- Exploitation requires implausible social engineering, a difficult race condition, guessing difficult-to guess data, or is otherwise unlikely.

Category

TestingXperts categorizes findings based on the security area to which those findings belong. This can help organizations identify gaps in secure development, deployment, patching, etc.

- **Authorization** - Related to authorization of users, and assessment of rights.
- **Auditing and Logging** - Related to auditing of actions, or logging of problems.
- **Authentication** - Related to the identification of users.
- **Configuration** - Related to security configurations of servers, devices, or software.
- **Cryptography** - Related to mathematical protections for data.
- **Data Exposure** - Related to unintended exposure of sensitive information.
- **Input Validation** - Related to improper reliance on the structure or values of data.
- **Denial of Service** - Related to causing system failure.
- **Error Reporting** - Related to the reporting of error conditions in a secure fashion.
- **Patching** - Related to keeping software up to date.
- **Session Management** - Related to the session management schema implemented in the application.
- **Timing** - Related to race conditions, locking, or order of operations.
- **Business Logic Bypass** – Related to bypassing the business logic implemented by the application.
- **Client Side Attack** – Related to execution of the code on the client, typically natively within a web browser or browser plugin

4. Conclusion

It has been observed that the application is vulnerable to publicly known attacks. We recommend mitigating the high-risk vulnerabilities on priority, followed by medium risk vulnerabilities, low-risk vulnerabilities and then informational vulnerabilities. Additionally, a full cycle of security testing must be carried out periodically or before a major release of the application.